

UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO
INTELIGÊNCIA ARTIFICIAL
TURMA 01

CARLOS ARTUR ALENCAR CRUZ
GUILHERME INÁCIO SANTOS PAES
IRWING FELIPE PEREIRA VIEIRA
RAPHAEL MATOS DA SILVA GONÇALVES
RENAN NUNES ANDRADE SAMPAIO

ALGORITMO GENÉTICO
SÃO CRISTÓVÃO - SE

22/02/2026

INTRODUÇÃO

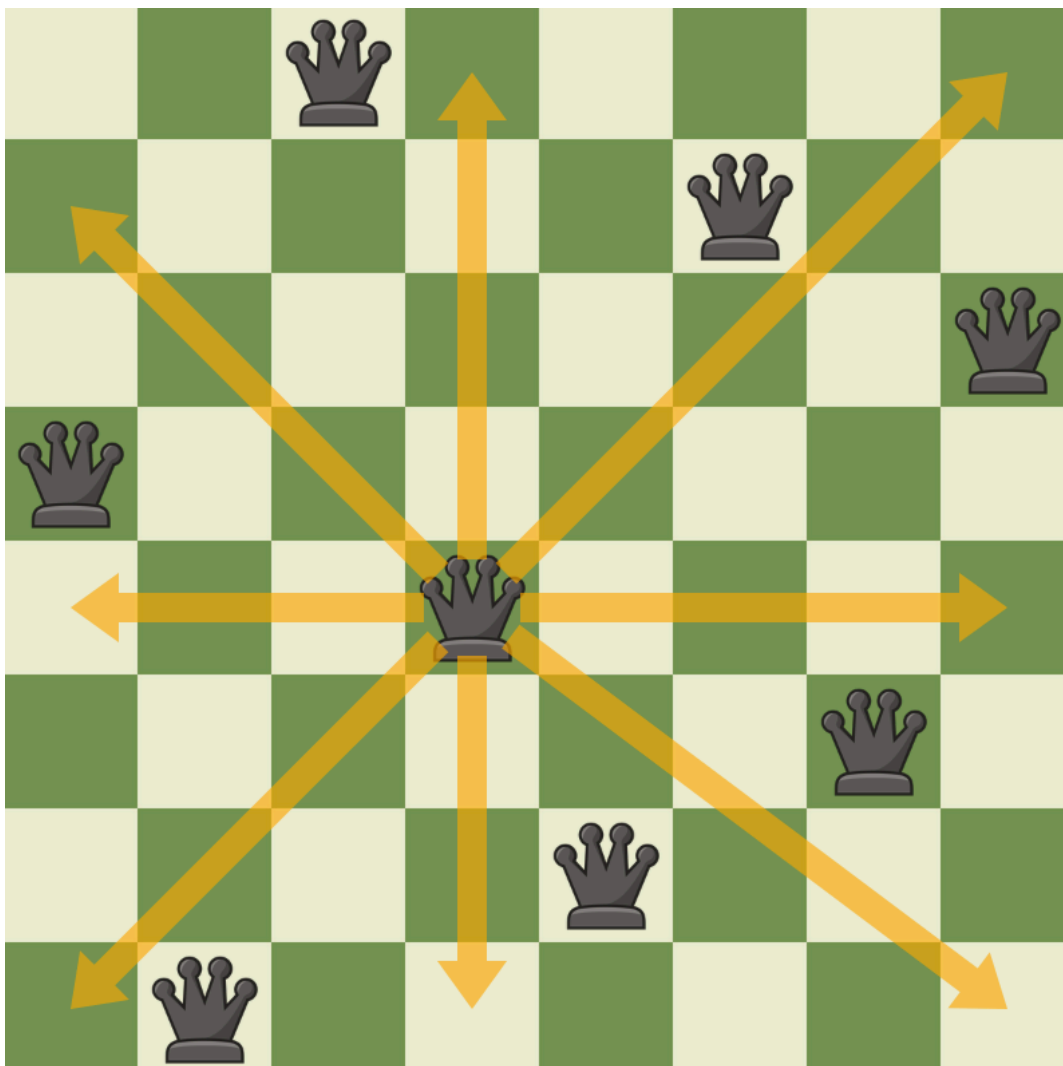
Esse relatório descreve a implementação de um Algoritmo Genético para resolução do problema clássico das N-Rainhas, um problema de busca e otimização combinatória.

DEFINIÇÃO DO PROBLEMA

O Problema das N-Rainhas consiste em posicionar N rainhas em um tabuleiro de proporção $N \times N$ de forma que nenhuma rainha ataque outra. Uma rainha ataca outras na mesma linha, coluna e diagonais.

Nesse projeto, cada solução candidata é representada por uma lista de tamanho N, onde o índice representa a coluna do tabuleiro e o valor representa a linha onde a rainha está posicionada. Essa modelagem elimina a possibilidade de conflitos de coluna, restando apenas os conflitos de linha e diagonais.

Exemplo: [4, 8, 1, 5, 7, 2, 6, 3]



MODELAGEM

O Problema das N-Rainhas pode ser observado como um problema de busca de estados combinatórios, onde o número de possíveis configurações aumenta exponencialmente de acordo com o valor de N. Considerando que cada uma das N colunas do tabuleiro pode conter uma rainha posicionada em qualquer uma das N linhas, o espaço de busca tem aproximadamente N^N possíveis configurações. Esse crescimento exponencial torna inviável a aplicação de métodos exaustivos. Nesse contexto, o Algoritmo Genético se destaca, pois é uma abordagem eficiente para solucionar problemas de otimização em espaços combinatórios.

O problema foi modelado como um problema de minimização, onde cada indivíduo representa um estado possível do tabuleiro e a função de fitness calcula o número de conflitos entre as rainhas. Assim, o objetivo do algoritmo é reduzir o valor da função de fitness e, se uma solução ótima for encontrada, a função de fitness será igual a zero.

PSEUDOCÓDIGO

As figuras a seguir apresentam as correspondências entre o pseudocódigo e a sua implementação, sendo que os trechos destacados com as mesmas cores mostram as partes equivalentes.

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
           FITNESS-FN, a function that measures the fitness of an individual

  repeat
    new_population  $\leftarrow$  empty set
    for i = 1 to SIZE(population) do
      x  $\leftarrow$  RANDOM-SELECTION(population, FITNESS-FN)
      y  $\leftarrow$  RANDOM-SELECTION(population, FITNESS-FN)
      child  $\leftarrow$  REPRODUCE(x, y)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to new_population
    population  $\leftarrow$  new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN



---


function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n  $\leftarrow$  LENGTH(x); c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

```
def genetic_algorithm(population, fitness, num_generations=20, mutation_rate=0.01):
    best_individual = random.choice(population)

    for i in tqdm(range(num_generations)):
        new_population = []
        for j in range(len(population)):
            x, y = random_selection(population, fitness)
            child = reproduce(x, y)
            if random.random() <= mutation_rate:
                child = mutate(child)
            new_population.append(child)

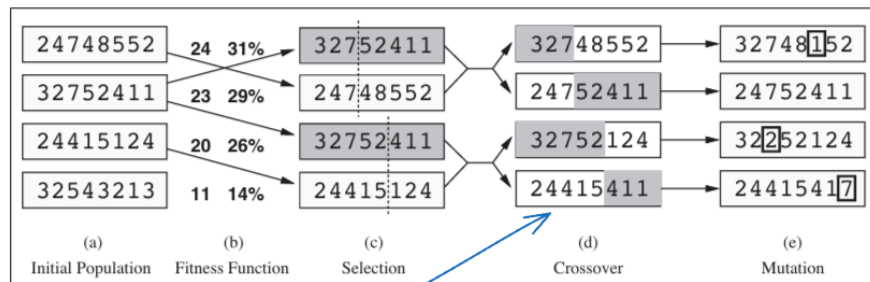
        population = new_population
        current_best_individual = selection(population, fitness)

        fitness_current = fitness(current_best_individual)
        fitness_best = fitness(best_individual)
        if fitness_current < fitness_best:
            best_individual = current_best_individual

        print(f'{best_individual}: {fitness_best}')

    return best_individual
```

```
def reproduce(x, y):
    n = len(x)
    c = random.randint(0, n-1)
    if random.random() < 0.5:
        child = x[:c] + y[c:]
    else:
        child = y[:c] + x[c:]
    return child
```



Tomei a liberdade de randomizar a parte do gene escolhida no crossover, como mostrado na figura.